Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 3, Lab 2

OpenMP: Sections, Single, and Critical

---

1 Ideas

- `#pragma omp parallel` — every thread runs that block.
- `#pragma omp parallel for` — gcc splits one `for` loop.
- **sections** — different named tasks, not slices of the same loop. Each section runs once. Order and thread ids can change.
- **single** — one thread runs that block. The others wait at the end of it, then all continue.
- **critical** — one thread at a time inside that { }. **Reduction** — private copies, then add (better for a sum). Critical — one key to the room (any shared update; a sum is slower).

Each section runs once. Extra threads wait. Two threads and four sections: some threads do two jobs in a row.

2 Commands

Terminal

```
nano omp_sections.c
gcc -fopenmp omp_sections.c -o sections
./sections

nano omp_single.c
gcc -fopenmp omp_single.c -o single
./single

nano omp_critical.c
gcc -fopenmp omp_critical.c -o critical
./critical
```

nano: Ctrl+O, Enter (save), Ctrl+X (quit).
Forgot `./` → command not found.
Forgot `-fopenmp` → error on `omp.h`, or still one thread.

3 Sections (named tasks)

Four tasks. Four threads. Each of A, B, C, D prints once.
C

```
#include <stdio.h>
#include <omp.h>

int main(void) {
    omp_set_num_threads(4);

    #pragma omp parallel
    {
```

```
        #pragma omp sections
        {
            #pragma omp section
            {
                printf("task A thread %d\n", omp_get_thread_num());
            }
            #pragma omp section
            {
                printf("task B thread %d\n", omp_get_thread_num());
            }
            #pragma omp section
            {
                printf("task C thread %d\n", omp_get_thread_num());
            }
            #pragma omp section
            {
                printf("task D thread %d\n", omp_get_thread_num());
            }
        }
    }

    return 0;
}
```

Run three times. Write which thread id did A, B, C, and D each time. Extra threads with no section left wait; they do not reprint A–D.

4 Single (one thread, then all continue)

`main before` and `main after` print once. `single work` prints once. `after single` prints once per thread.

C

```
#include <stdio.h>
#include <omp.h>

int main(void) {
    omp_set_num_threads(4);

    printf("main before\n");

    #pragma omp parallel
    {
        #pragma omp single
        {
            printf("single work thread %d\n", omp_get_thread_num());
        }

        printf("after single thread %d\n", omp_get_thread_num());
    }

    printf("main after\n");
    return 0;
}
```

If `single work` prints four times, the `single` directive is missing.

5 Critical (safe shared update)

Expected total for $N = 100000$:

$$\frac{N(N-1)}{2} = 4999950000$$

C

```
#include <stdio.h>
#include <omp.h>

#define N 100000

int main(void) {
    long i;
    long sum = 0;

    omp_set_num_threads(4);

    #pragma omp parallel for
    for (i = 0; i < N; i++) {
        #pragma omp critical
        {
            sum += i;
        }
    }

    printf("sum = %ld\n", sum);
    printf("expected = %ld\n", (long)N * (N - 1) / 2);
    return 0;
}
```

Run twice. Printed `sum` must equal expected both times. This is usually slower than reduction: threads take turns on `sum`.